

```
In [ ]: import marimo as mo
```

VQ-VAE Tokenizer Evaluation

Protein / Ligand VQ-VAE の学習後の定量的性能評価ノートブック。

評価項目:

1. 再構成誤差 (MSE) — 全体 & 次元別
2. Codebook 利用率 & Perplexity
3. Original vs Reconstructed の散布図
4. Codebook 使用頻度の分布
5. 潜在空間の可視化 (t-SNE)

```
In [ ]: import sys
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import torch

# Add project root to path (resolve from this file, not cwd)
project_root = Path(__file__).resolve().parent.parent
if str(project_root) not in sys.path:
    sys.path.insert(0, str(project_root))

from src.config import CrossDockedConfig, VQVAETrainingConfig
from src.data.descriptors import ComplexDescriptorDataModule
from src.model.vqvae_module import VQVAEModule

# '%matplotlib inline' command supported automatically in marimo
plt.rcParams["figure.dpi"] = 120
```

1. Load checkpoint and data

```
In [ ]: # --- Checkpoint path (edit here) ---
# Best checkpoint is typically the one with lowest val/protein_recon.
# List available checkpoints:
ckpt_dir = project_root / "checkpoints"
if ckpt_dir.exists():
    ckpts = sorted(ckpt_dir.rglob("*.ckpt"))
    for _p in ckpts:
        print(_p.relative_to(project_root))
else:
    print("No checkpoints/ directory found. Check wandb run directory.")
    ckpts = sorted(project_root.rglob("*.ckpt"))
    for _p in ckpts[-5:]:
        print(_p.relative_to(project_root))
```

```
checkpoints/vqvae-epoch=02-val/protein_recon=0.7866.ckpt
checkpoints/vqvae-epoch=03-val/protein_recon=0.6128.ckpt
checkpoints/vqvae-epoch=04-val/protein_recon=0.4670.ckpt
```

```
In [ ]: # Load the best checkpoint (pick the last one = lowest val loss)
CKPT_PATH = "/home/sakano/git/pocket-conditioned-ligand-gen/pocket-ligand-vqvae/42tfb6kx/checkpoints/vqvae-epoch=19-val/protein_recon=0.1512.ckpt"
print(f"Loading: {CKPT_PATH}")

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
module = VQVAEModule.load_from_checkpoint(str(CKPT_PATH), map_location=device)
module.eval()
module.to(device)

protein_vqvae = module.protein_vqvae
ligand_vqvae = module.ligand_vqvae
print(f"Device: {device}")
```

```
Loading: /home/sakano/git/pocket-conditioned-ligand-gen/pocket-ligand-vqvae/42tfb6kx/checkpoints/vqvae-epoch=19-val/protein_recon=0.1512.ckpt
```

```
/home/sakano/git/pocket-conditioned-ligand-gen/.venv/lib/python3.12/site-packages/torch/nn/modules/transformer.py:392: UserWarning: enable_nested_tensor is True, but self.use_nested_tensor is False because encoder_layer.norm_first was True
```

```
warnings.warn(
```

```
Device: cuda
```

```
In [ ]: # Load cached descriptors and prepare validation split
config = VQVAETrainingConfig()
data_config = CrossDockedConfig(data_dir=project_root / "data")
dm = ComplexDescriptorDataModule(config, data_config)
dm.setup()

protein_val = dm.protein_val.to(device)
# Ligand val is now a list of per-molecule tensors (for Transformer sequence)
ligand_val_molecules = [mol.to(device) for mol in dm.ligand_val]
ligand_val_flat = torch.cat(ligand_val_molecules)

# Also load normalization stats for de-normalization
norm_stats = torch.load(
    project_root / "data" / "descriptor_cache" / "normalization_stats.pt",
    weights_only=True,
)
print(f"Protein val: {protein_val.shape}")
print(
    f"Ligand val: {len(ligand_val_molecules)} molecules, {ligand_val_flat.shape}"
)
```

```
Protein val: torch.Size([4719527, 9])
```

```
Ligand val: 156428 molecules, 4659526 atoms total
```

```
In [ ]: # Run inference on validation set
@torch.no_grad()
def run_vqvae_protein(
    model: "ProteinStructureVQVAE", data: "Tensor", batch_size: int = 4096
):
    """Run protein VQ-VAE on flat residue data."""
    all_recon, all_indices, all_z = [], [], []
```

```

for i in range(0, len(data), batch_size):
    batch = data[i : i + batch_size]
    z = model.encoder(batch)
    quantized, indices, _ = model.codebook(z)
    recon = model.decoder(quantized)
    all_recon.append(recon)
    all_indices.append(indices)
    all_z.append(z)
return torch.cat(all_recon), torch.cat(all_indices), torch.cat(all_z)

@torch.no_grad()
def run_vqvae_ligand(model: "LigandVQVAE", molecules: "list[Tensor]"):
    """Run ligand VQ-VAE per-molecule with transformer context."""
    all_recon, all_indices, all_z = [], [], []
    for mol in molecules:
        x_seq = mol.unsqueeze(0) # (1, N, D)
        h = model.input_proj(x_seq) + model.pos_encoding[: mol.shape[0]]
        h = model.transformer_encoder(h)
        z = model.latent_proj(h).squeeze(0) # (N, latent_dim)
        quantized, indices, _ = model.codebook(z)
        q_seq = model.latent_unproj(quantized).unsqueeze(0)
        dec_in = q_seq + model.pos_encoding[: mol.shape[0]]
        dec_out = model.transformer_decoder(dec_in)
        recon = model.output_proj(dec_out).squeeze(0) # (N, D)
        all_recon.append(recon)
        all_indices.append(indices)
        all_z.append(z)
    return torch.cat(all_recon), torch.cat(all_indices), torch.cat(all_z)

prot_recon, prot_indices, prot_z = run_vqvae_protein(protein_vqvae, protein_val_mols)
lig_recon, lig_indices, lig_z = run_vqvae_ligand(ligand_vqvae, ligand_val_mols)

print("Inference done.")
print(
    f"Protein: recon {prot_recon.shape}, indices {prot_indices.shape}, z {prot_z.shape}"
)
print(
    f"Ligand: recon {lig_recon.shape}, indices {lig_indices.shape}, z {lig_z.shape}"
)

```

Inference done.

Protein: recon torch.Size([4719527, 9]), indices torch.Size([4719527]), z to torch.Size([4719527, 16])

Ligand: recon torch.Size([4659526, 4]), indices torch.Size([4659526]), z to torch.Size([4659526, 8])

2. Reconstruction Error (MSE)

正規化空間と元スケールの両方で再構成誤差を評価する。

```

In [ ]: def compute_mse_metrics(
        original: "Tensor",
        reconstructed: "Tensor",
        name: str,
        norm_mean: "Tensor",

```

```

norm_std: "Tensor",
):
    """Compute MSE in both normalized and original scale."""
    orig_np = original.cpu().numpy()
    recon_np = reconstructed.cpu().numpy()

    # Normalized space
    mse_norm = np.mean((orig_np - recon_np) ** 2)
    per_dim_mse_norm = np.mean((orig_np - recon_np) ** 2, axis=0)

    # De-normalize to original scale
    mean_np = norm_mean.numpy()
    std_np = norm_std.numpy()
    orig_denorm = orig_np * std_np + mean_np
    recon_denorm = recon_np * std_np + mean_np
    mse_orig = np.mean((orig_denorm - recon_denorm) ** 2)
    per_dim_mse_orig = np.mean((orig_denorm - recon_denorm) ** 2, axis=0)

    print(f"=== {name} VQ-VAE ===")
    print(f" Overall MSE (normalized): {mse_norm:.6f}")
    print(f" Overall MSE (original): {mse_orig:.6f}")
    print(f" Overall RMSE (original): {np.sqrt(mse_orig):.6f}")
    print()

    return {
        "mse_norm": mse_norm,
        "per_dim_mse_norm": per_dim_mse_norm,
        "mse_orig": mse_orig,
        "per_dim_mse_orig": per_dim_mse_orig,
        "orig_denorm": orig_denorm,
        "recon_denorm": recon_denorm,
    }

prot_metrics = compute_mse_metrics(
    protein_val,
    prot_recon,
    "Protein",
    norm_stats["protein_mean"],
    norm_stats["protein_std"],
)
lig_metrics = compute_mse_metrics(
    ligand_val_flat,
    lig_recon,
    "Ligand",
    norm_stats["ligand_mean"],
    norm_stats["ligand_std"],
)

```

```

=== Protein VQ-VAE ===
Overall MSE (normalized):  0.151219
Overall MSE (original):    1.685398
Overall RMSE (original):   1.298229

=== Ligand VQ-VAE ===
Overall MSE (normalized):  0.002934
Overall MSE (original):    0.001459
Overall RMSE (original):   0.038194

```

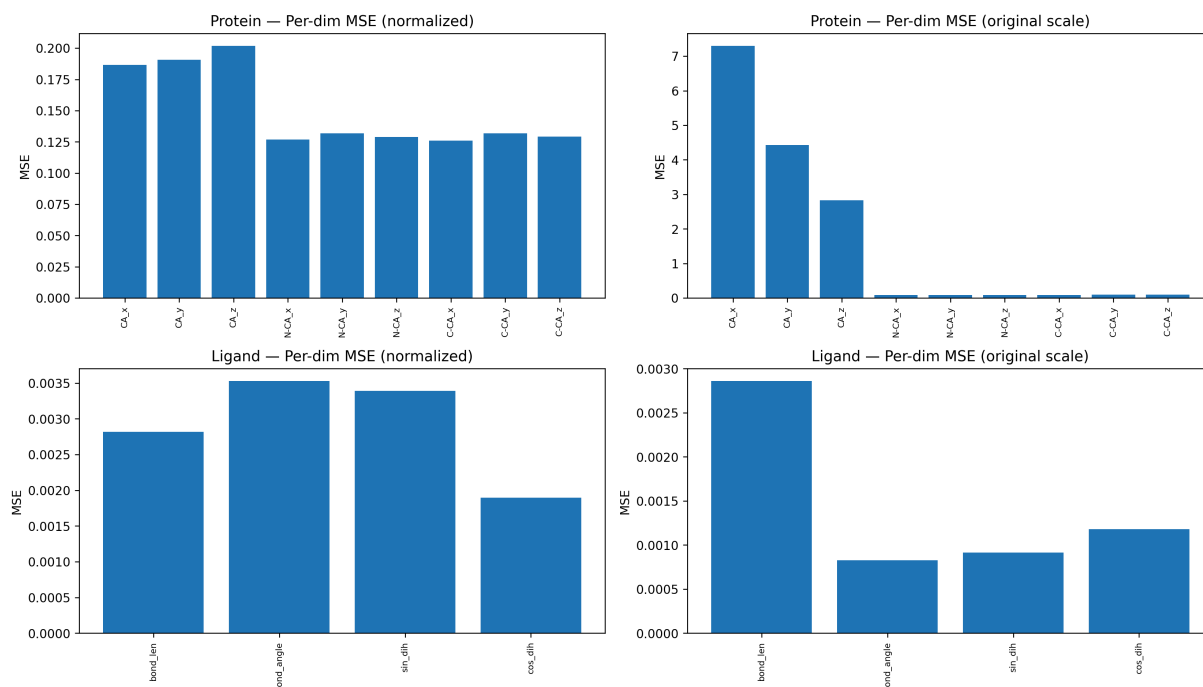
2.1 Per-dimension MSE

Protein: dim 0–2 = CA position, dim 3–5 = N-CA offset, dim 6–8 = C-CA offset (canonical frame) Ligand: dim 0 = bond length, dim 1 = bond angle, dim 2–3 = sin/cos dihedral

```

In [ ]: _fig, _axes = plt.subplots(2, 2, figsize=(14, 8))
prot_dim_labels = [
    "CA_x",
    "CA_y",
    "CA_z",
    "N-CA_x",
    "N-CA_y",
    "N-CA_z",
    "C-CA_x",
    "C-CA_y",
    "C-CA_z",
]
# Protein – normalized
_axes[0, 0].bar(range(9), prot_metrics["per_dim_mse_norm"])
_axes[0, 0].set_xticks(range(9), prot_dim_labels, rotation=90, fontsize=7)
_axes[0, 0].set_title("Protein – Per-dim MSE (normalized)")
_axes[0, 0].set_ylabel("MSE")
_axes[0, 1].bar(range(9), prot_metrics["per_dim_mse_orig"])
_axes[0, 1].set_xticks(range(9), prot_dim_labels, rotation=90, fontsize=7)
# Protein – original scale
_axes[0, 1].set_title("Protein – Per-dim MSE (original scale)")
_axes[0, 1].set_ylabel("MSE")
lig_dim_labels = ["bond_len", "bond_angle", "sin_dih", "cos_dih"]
_axes[1, 0].bar(range(4), lig_metrics["per_dim_mse_norm"])
_axes[1, 0].set_xticks(range(4), lig_dim_labels, rotation=90, fontsize=7)
# Ligand – normalized
_axes[1, 0].set_title("Ligand – Per-dim MSE (normalized)")
_axes[1, 0].set_ylabel("MSE")
_axes[1, 1].bar(range(4), lig_metrics["per_dim_mse_orig"])
_axes[1, 1].set_xticks(range(4), lig_dim_labels, rotation=90, fontsize=7)
_axes[1, 1].set_title("Ligand – Per-dim MSE (original scale)")
_axes[1, 1].set_ylabel("MSE")
# Ligand – original scale
_fig.tight_layout()
_fig

```



3. Codebook Utilization & Perplexity

理想的なcodebookは全コードが均等に使われる。

- **Utilization:** 使用されたコード数 / 全コード数 (1.0が理想)
- **Perplexity:** $\exp(\text{entropy})$ 。均等利用時は codebook_size と一致

```
In [ ]: def codebook_stats(indices: "Tensor", codebook_size: int, name: str):
    """Compute and print codebook utilization metrics."""
    idx_np = indices.cpu().numpy()
    unique = np.unique(idx_np)
    utilization = len(unique) / codebook_size

    counts = np.bincount(idx_np, minlength=codebook_size).astype(float)
    probs = counts / counts.sum()
    probs_nonzero = probs[probs > 0]
    entropy = -np.sum(probs_nonzero * np.log(probs_nonzero))
    perplexity = np.exp(entropy)

    # Max perplexity = codebook_size (uniform distribution)
    max_perplexity = codebook_size
    normalized_perplexity = perplexity / max_perplexity

    print(f"=== {name} Codebook ===")
    print(f"  Codebook size:      {codebook_size}")
    print(f"  Active codes:       {len(unique)}")
    print(f"  Utilization:        {utilization:.4f}")
    print(f"  Perplexity:         {perplexity:.1f} / {max_perplexity}")
    print(f"  Normalized perplexity: {normalized_perplexity:.4f}")
    print(f"  Dead codes:         {codebook_size - len(unique)}")
    print()
    return counts
```

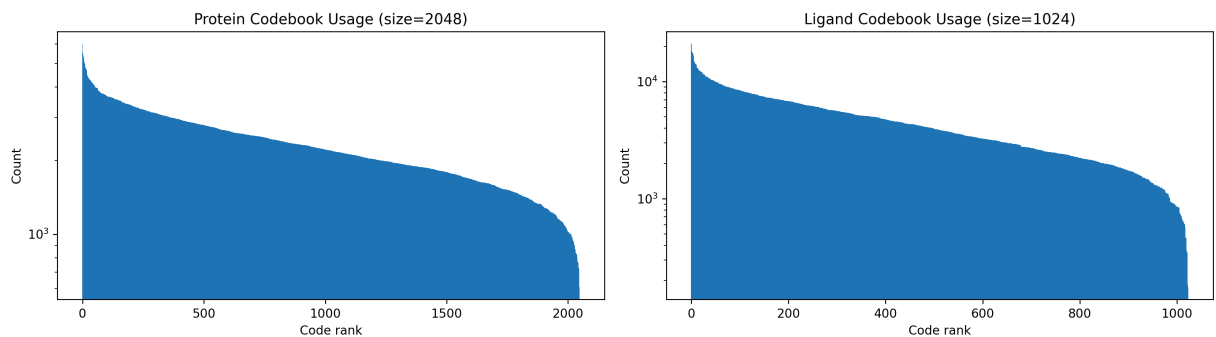
```
prot_counts = codebook_stats(prot_indices, config.protein.codebook_size, "Protein")
lig_counts = codebook_stats(lig_indices, config.ligand.codebook_size, "Ligand")
```

```
=== Protein Codebook ===
Codebook size:      2048
Active codes:       2048
Utilization:        1.0000
Perplexity:         1936.9 / 2048
Normalized perplexity: 0.9457
Dead codes:         0
```

```
=== Ligand Codebook ===
Codebook size:      1024
Active codes:       1024
Utilization:        1.0000
Perplexity:         851.2 / 1024
Normalized perplexity: 0.8313
Dead codes:         0
```

3.1 Codebook usage distribution

```
In [ ]: _fig, _axes = plt.subplots(1, 2, figsize=(14, 4))
prot_sorted = np.sort(prot_counts)[::-1]
# Protein codebook usage - sorted descending
_axes[0].bar(range(len(prot_sorted)), prot_sorted, width=1.0)
_axes[0].set_xlabel("Code rank")
_axes[0].set_ylabel("Count")
_axes[0].set_title(f"Protein Codebook Usage (size={config.protein.codebook_size})")
_axes[0].set_yscale("log")
lig_sorted = np.sort(lig_counts)[::-1]
_axes[1].bar(range(len(lig_sorted)), lig_sorted, width=1.0)
# Ligand codebook usage - sorted descending
_axes[1].set_xlabel("Code rank")
_axes[1].set_ylabel("Count")
_axes[1].set_title(f"Ligand Codebook Usage (size={config.ligand.codebook_size})")
_axes[1].set_yscale("log")
_fig.tight_layout()
_fig
```



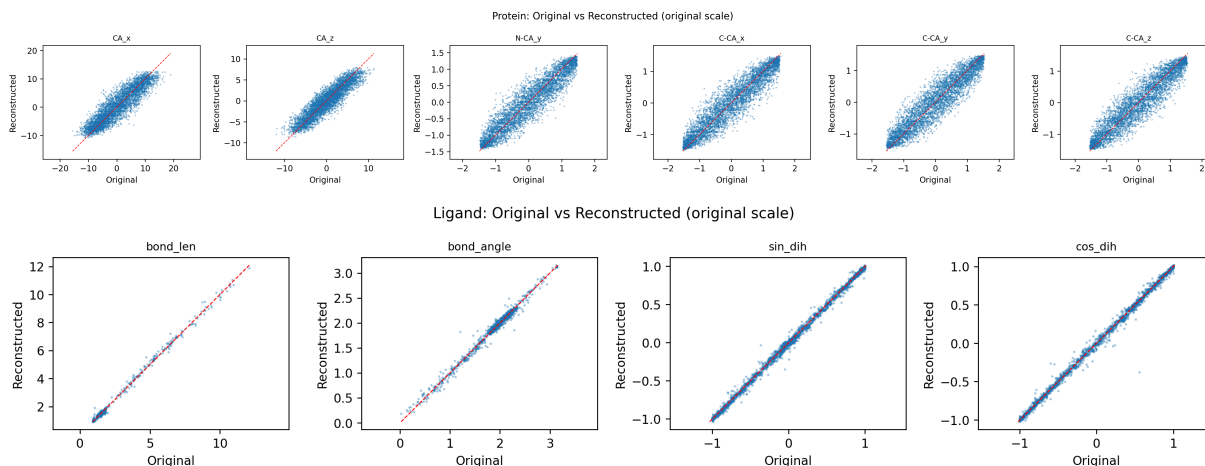
4. Original vs Reconstructed scatter plots

各次元で元の値 vs 再構成値をプロットし、対角線からのずれを確認する。元スケールで表示。

```
In [ ]: def scatter_orig_vs_recon(
    orig: "ndarray",
    recon: "ndarray",
    dim_labels: list[str],
    title: str,
    max_points: int = 5000,
):
    """Scatter plot of original vs reconstructed for selected dimensions."""
    n_dims = orig.shape[1]
    max_inline_dims = 8
    if n_dims <= max_inline_dims:
        selected = list(range(n_dims))
    else:
        selected = [
            0,
            n_dims // 4,
            n_dims // 2,
            3 * n_dims // 4,
            n_dims - 2,
            n_dims - 1,
        ]
    n_plots = len(selected)
    _fig, _axes = plt.subplots(1, n_plots, figsize=(3.5 * n_plots, 3.2))
    if n_plots == 1:
        _axes = [_axes]
    rng = np.random.default_rng(42)
    idx = rng.choice(len(orig), min(max_points, len(orig)), replace=False)
    for ax, dim in zip(_axes, selected, strict=False):
        ax.scatter(orig[idx, dim], recon[idx, dim], s=1, alpha=0.3)
        lo = min(orig[idx, dim].min(), recon[idx, dim].min())
        hi = max(orig[idx, dim].max(), recon[idx, dim].max())
        ax.plot([lo, hi], [lo, hi], "r--", linewidth=0.8)
        ax.set_xlabel("Original")
        ax.set_ylabel("Reconstructed")
        ax.set_title(dim_labels[dim], fontsize=9)
        ax.set_aspect("equal", adjustable="datalim") # Subsample for plotti
    _fig.suptitle(title, fontsize=12)
    _fig.tight_layout()
    plt.show()

scatter_orig_vs_recon(
    prot_metrics["orig_denorm"],
    prot_metrics["recon_denorm"],
    prot_dim_labels,
    "Protein: Original vs Reconstructed (original scale)",
)

scatter_orig_vs_recon(
    lig_metrics["orig_denorm"],
    lig_metrics["recon_denorm"],
    lig_dim_labels,
    "Ligand: Original vs Reconstructed (original scale)",
)
```

5. Reconstruction error distribution

サンプルごとの再構成誤差の分布を確認し、外れ値の存在を把握する。

```
In [ ]: _fig, _axes = plt.subplots(1, 2, figsize=(12, 4))
prot_per_sample = np.mean(
    (protein_val.cpu().numpy() - prot_recon.cpu().numpy()) ** 2, axis=1
)
lig_per_sample = np.mean(
    (ligand_val_flat.cpu().numpy() - lig_recon.cpu().numpy()) ** 2, axis=1
)
_axes[0].hist(prot_per_sample, bins=100, edgecolor="none", alpha=0.8)
_axes[0].axvline(
    np.median(prot_per_sample),
    color="r",
    linestyle="--",
    label=f"median={np.median(prot_per_sample):.4f}",
)
_axes[0].axvline(
    np.mean(prot_per_sample),
    color="orange",
    linestyle="--",
    label=f"mean={np.mean(prot_per_sample):.4f}",
)
_axes[0].set_xlabel("Per-sample MSE")
_axes[0].set_ylabel("Count")
_axes[0].set_title("Protein - Per-sample MSE distribution")
_axes[0].legend(fontsize=8)
_axes[1].hist(lig_per_sample, bins=100, edgecolor="none", alpha=0.8)
_axes[1].axvline(
    np.median(lig_per_sample),
    color="r",
    linestyle="--",
    label=f"median={np.median(lig_per_sample):.4f}",
)
_axes[1].axvline(
    np.mean(lig_per_sample),
    color="orange",
    linestyle="--",

```

```

        label=f"mean={np.mean(lig_per_sample):.4f}",
    )
    _axes[1].set_xlabel("Per-sample MSE")
    _axes[1].set_ylabel("Count")
    _axes[1].set_title("Ligand - Per-sample MSE distribution")
    _axes[1].legend(fontsize=8)
    _fig.tight_layout()
    for name, mse_arr in [("Protein", prot_per_sample), ("Ligand", lig_per_sample)]:
        print(f"{name} per-sample MSE percentiles:")
        for _p in [50, 75, 90, 95, 99]:
            print(f"    P{_p}: {np.percentile(mse_arr, _p):.6f}")
        print()
    _fig

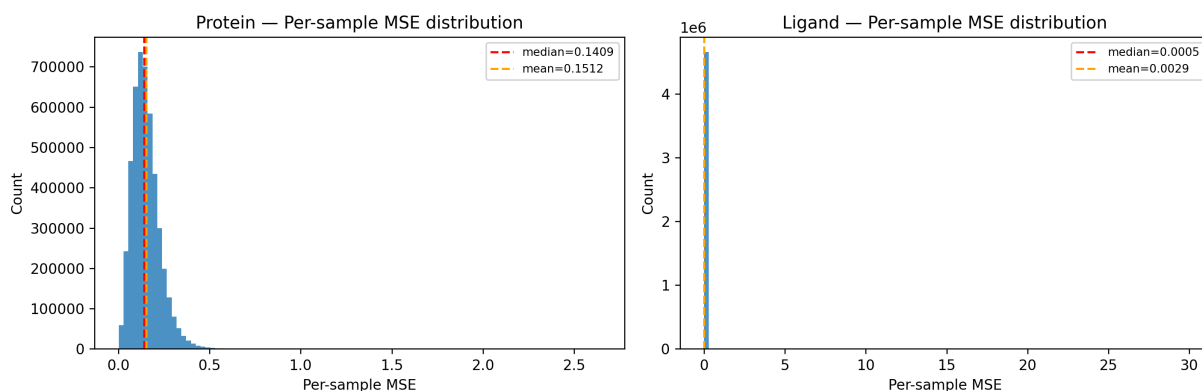
```

Protein per-sample MSE percentiles:

P50: 0.140883
 P75: 0.191689
 P90: 0.247865
 P95: 0.288209
 P99: 0.383333

Ligand per-sample MSE percentiles:

P50: 0.000517
 P75: 0.001551
 P90: 0.004138
 P95: 0.008128
 P99: 0.032987



5.5 3D Reconstruction RMSE (Å)

記述子空間での再構成誤差ではなく、実際の3次元座標に復元した上でのRMSEを評価する。

- **Protein:** backbone (N, CA, C) 座標の RMSE
- **Ligand:** 重原子座標の RMSE

複数の複合体をサンプリングし、VQ-VAE の encode → decode → 記述子逆変換 → 3D座標のパイプライン全体の精度を確認する。

```

In [ ]: from src.config import PocketExtractionConfig
        from src.data.descriptors import _parse_types_file
        from src.tokenizers.ligand import LigandDescriptor, parse_sdf
        from src.tokenizers.protein import PocketDescriptor, extract_pocket

pocket_config = PocketExtractionConfig()
protein_desc_calc = PocketDescriptor()
ligand_desc_calc = LigandDescriptor()
types_file = project_root / "data" / "types" / "cdonly_it2_tt_v1.3_0_train0.
pairs = _parse_types_file(types_file)
crossdocked_dir = project_root / "data" / "CrossDocked2020"
prot_mean = norm_stats["protein_mean"].to(device)
prot_std = norm_stats["protein_std"].to(device)
lig_mean = norm_stats["ligand_mean"].to(device)
lig_std = norm_stats["ligand_std"].to(device)
N_SAMPLES = 200
rng = np.random.default_rng(42)
sample_indices = rng.choice(
    len(pairs), size=min(N_SAMPLES * 5, len(pairs)), replace=False
)
prot_rmse_list = []
lig_rmse_list = []
n_done = 0
for idx in sample_indices:
    if n_done >= N_SAMPLES:
        break
    rec_rel, lig_rel = pairs[idx]
    rec_path = crossdocked_dir / rec_rel
    lig_path = crossdocked_dir / lig_rel
    if not rec_path.exists() or not lig_path.exists():
        continue
    try:
        molecules = parse_sdf(lig_path)
        if not molecules:
            continue
        mol = molecules[0]
        lig_coords_orig = np.array(
            [(a[1], a[2], a[3]) for a in mol["atoms"] if a[0] != "H"],
            dtype=np.float32,
        )
        pocket_result = extract_pocket(rec_path, lig_coords_orig, pocket_config)
        if pocket_result is None:
            continue
        backbone_coords_orig, _seq = pocket_result
        prot_desc, prot_meta = protein_desc_calc.compute(backbone_coords_orig, _seq)
        pocket_frame = (prot_meta["centroid"], prot_meta["rotation"])
        prot_t = torch.from_numpy(prot_desc).to(device)
        prot_norm = (prot_t - prot_mean) / prot_std
        with torch.no_grad():
            pi = protein_vqvae.encode(prot_norm)
            prot_recon_norm = protein_vqvae.decode(pi)
        prot_recon_desc = (prot_recon_norm * prot_std + prot_mean).cpu().numpy()
        backbone_recon = PocketDescriptor.descriptor_to_backbone_coords(
            prot_recon_desc, prot_meta
        )

```

```

prot_rmse = np.sqrt(np.mean((backbone_coords_orig - backbone_recon)
prot_rmse_list.append(prot_rmse)
lig_desc, _elements, lig_meta = ligand_desc_calc.compute(
    mol["atoms"], mol["bonds"], pocket_frame=pocket_frame
)
if len(lig_desc) == 0:
    continue
lig_t = torch.from_numpy(lig_desc).to(device)
lig_norm = (lig_t - lig_mean) / lig_std
with torch.no_grad():
    li = ligand_vqvae.encode(lig_norm)
    lig_recon_norm = ligand_vqvae.decode(li)
lig_recon_desc = (lig_recon_norm * lig_std + lig_mean).cpu().numpy()
lig_coords_recon = LigandDescriptor.descriptor_to_coords(
    lig_recon_desc, lig_meta, pocket_frame=pocket_frame
)
heavy_atoms = [a for a in mol["atoms"] if a[0] != "H"]
lig_coords_orig_arr = np.array(
    [(a[1], a[2], a[3]) for a in heavy_atoms], dtype=np.float64
)
lig_rmse = np.sqrt(np.mean((lig_coords_orig_arr - lig_coords_recon)
lig_rmse_list.append(lig_rmse)
n_done += 1
except Exception: # noqa: BLE001, S112
    continue
prot_rmse_arr = np.array(prot_rmse_list)
lig_rmse_arr = np.array(lig_rmse_list)
print(f"Evaluated {len(prot_rmse_arr)} complexes")
print()
print("Protein backbone RMSE (Å):")
print(f"  Mean:   {prot_rmse_arr.mean():.4f}")
print(f"  Median: {np.median(prot_rmse_arr):.4f}")
print(f"  Std:    {prot_rmse_arr.std():.4f}")
print()
print("Ligand heavy-atom RMSE (Å):")
print(f"  Mean:   {lig_rmse_arr.mean():.4f}")
print(f"  Median: {np.median(lig_rmse_arr):.4f}")
print(f"  Std:    {lig_rmse_arr.std():.4f}")
_fig, _axes = plt.subplots(1, 2, figsize=(12, 4))
_axes[0].hist(prot_rmse_arr, bins=50, edgecolor="none", alpha=0.8)
_axes[0].axvline(
    np.median(prot_rmse_arr),
    color="r",
    linestyle="--",
    label=f"median={np.median(prot_rmse_arr):.3f} Å",
)
_axes[0].axvline(
    prot_rmse_arr.mean(),
    color="orange",
    linestyle="--",
    label=f"mean={prot_rmse_arr.mean():.3f} Å",
)
_axes[0].set_xlabel("RMSE (Å)")
_axes[0].set_ylabel("Count")
_axes[0].set_title("Protein backbone 3D RMSE")
_axes[0].legend(fontsize=8)

```

```

_axes[1].hist(lig_rmse_arr, bins=50, edgecolor="none", alpha=0.8)
_axes[1].axvline(
    np.median(lig_rmse_arr),
    color="r",
    linestyle="--",
    label=f"median={np.median(lig_rmse_arr):.3f} Å",
)
_axes[1].axvline(
    lig_rmse_arr.mean(),
    color="orange",
    linestyle="--",
    label=f"mean={lig_rmse_arr.mean():.3f} Å",
)
_axes[1].set_xlabel("RMSE (Å)")
_axes[1].set_ylabel("Count")
_axes[1].set_title("Ligand heavy-atom 3D RMSE")
_axes[1].legend(fontsize=8)
_fig.tight_layout()
_fig

```

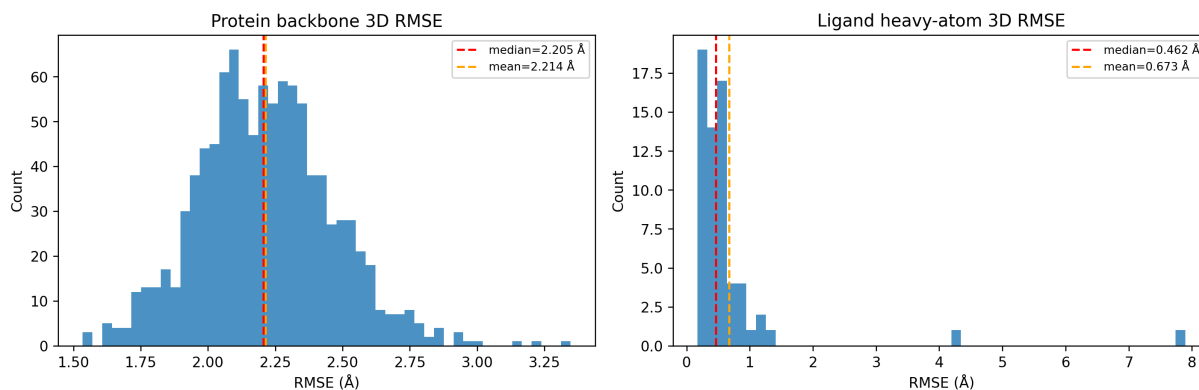
Evaluated 1000 complexes

Protein backbone RMSE (Å):

Mean: 2.2140
Median: 2.2053
Std: 0.2489

Ligand heavy-atom RMSE (Å):

Mean: 0.6731
Median: 0.4619
Std: 1.0542



6. Latent space visualization (t-SNE)

Codebook ベクトルとエンコーダ出力を t-SNE で2次元に射影し、量子化の質を確認する。

```

In [ ]: import torch.nn.functional as F
        from sklearn.manifold import TSNE

        def plot_latent_tsne(
            encoder_z: "Tensor",
            codebook: "EMACodebook",
            _codebook_size: int,

```

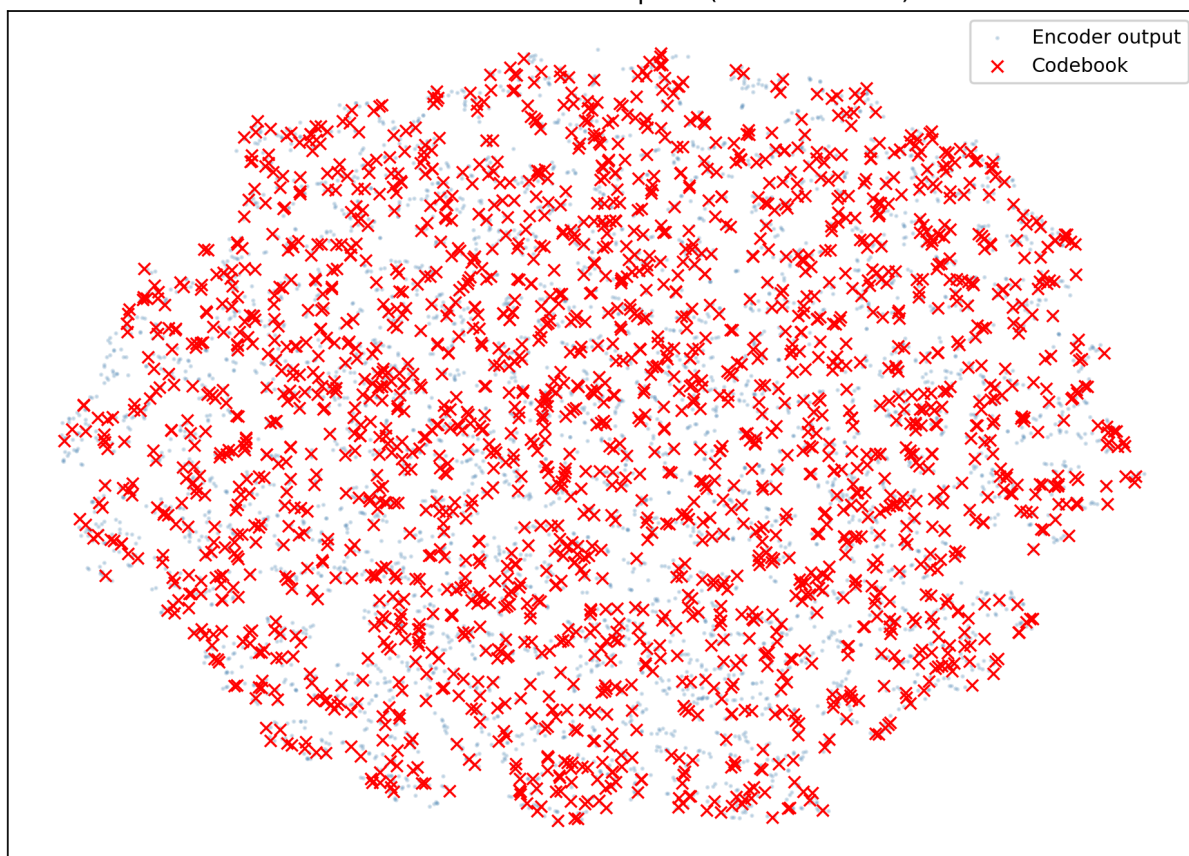
```

name: str,
n_samples: int = 5000,
) -> None:
    """t-SNE visualization of encoder outputs and codebook vectors."""
    z_norm = F.normalize(encoder_z, p=2, dim=-1).cpu().numpy()
    cb_norm = (
        F.normalize(codebook.embedding, p=2, dim=-1).cpu().detach().numpy()
    ) # L2 normalize (same as codebook forward)
    rng = np.random.default_rng(42)
    idx = rng.choice(len(z_norm), min(n_samples, len(z_norm)), replace=False)
    z_sub = z_norm[idx]
    combined = np.vstack([z_sub, cb_norm]) # Subsample encoder outputs
    tsne = TSNE(
        n_components=2, random_state=42, perplexity=min(30, len(combined)) //
    )
    embedded = tsne.fit_transform(combined)
    z_emb = embedded[: len(z_sub)]
    cb_emb = embedded[len(z_sub) :]
    _fig, ax = plt.subplots(figsize=(8, 6)) # Combine for t-SNE
    ax.scatter(
        z_emb[:, 0],
        z_emb[:, 1],
        s=1,
        alpha=0.2,
        c="steelblue",
        label="Encoder output",
    )
    ax.scatter(
        cb_emb[:, 0],
        cb_emb[:, 1],
        s=30,
        c="red",
        marker="x",
        linewidths=1,
        label="Codebook",
    )
    ax.set_title(f"{name} - t-SNE of latent space (L2-normalized)")
    ax.legend(fontsize=9)
    ax.set_xticks([])
    ax.set_yticks([])
    _fig.tight_layout()
    plt.show()

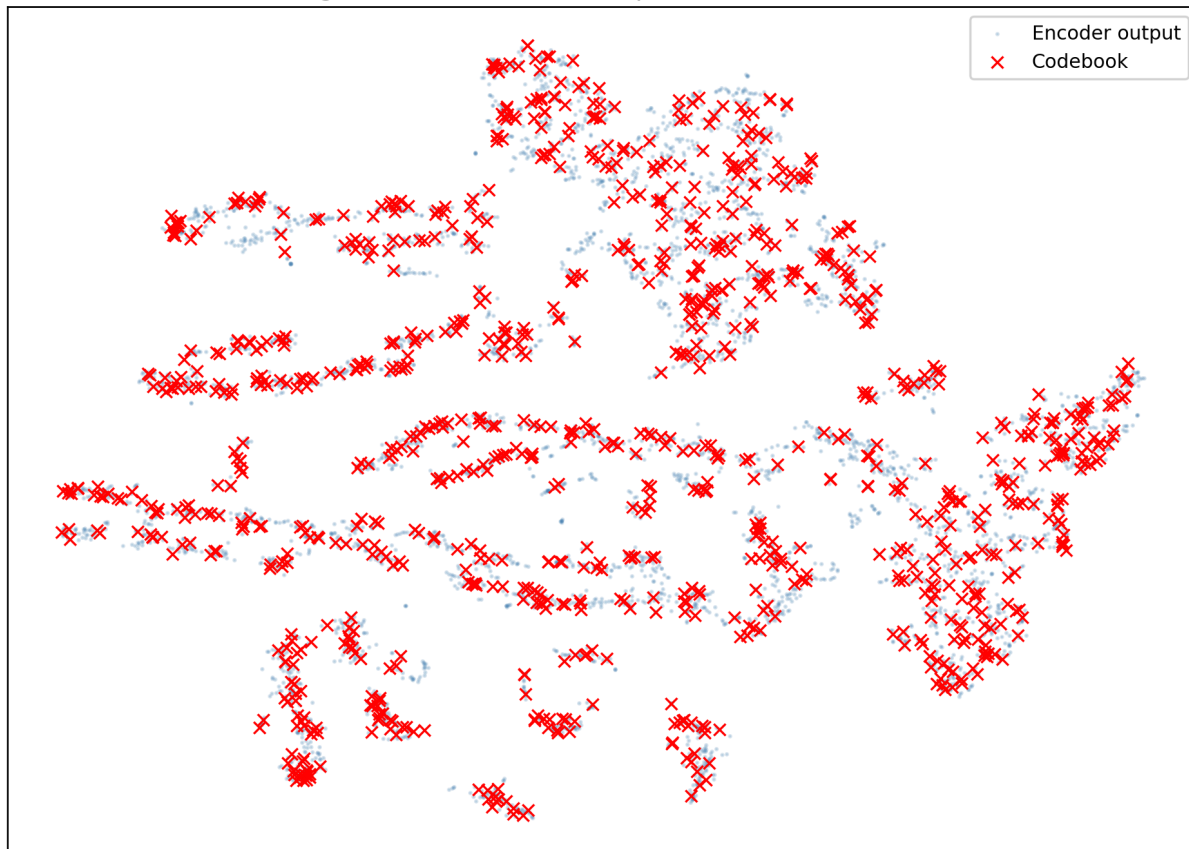
plot_latent_tsne(
    prot_z, protein_vqvae.codebook, config.protein.codebook_size, "Protein"
)
plot_latent_tsne(
    lig_z, ligand_vqvae.codebook, config.ligand.codebook_size, "Ligand"
)

```

Protein — t-SNE of latent space (L2-normalized)



Ligand — t-SNE of latent space (L2-normalized)



7. Summary table

```
In [ ]: import pandas as pd

prot_idx_np = prot_indices.cpu().numpy()
lig_idx_np = lig_indices.cpu().numpy()

prot_probs = np.bincount(
    prot_idx_np, minlength=config.protein.codebook_size
).astype(float)
prot_probs = prot_probs / prot_probs.sum()
prot_probs_nz = prot_probs[prot_probs > 0]
prot_ppl = np.exp(-np.sum(prot_probs_nz * np.log(prot_probs_nz)))

lig_probs = np.bincount(lig_idx_np, minlength=config.ligand.codebook_size).a
    float
)
lig_probs = lig_probs / lig_probs.sum()
lig_probs_nz = lig_probs[lig_probs > 0]
lig_ppl = np.exp(-np.sum(lig_probs_nz * np.log(lig_probs_nz)))

summary = pd.DataFrame(
    {
        "Metric": [
            "Codebook size",
            "Latent dim",
            "Val MSE (normalized)",
            "Val MSE (original scale)",
            "Val RMSE (original scale)",
            "Active codes",
            "Utilization",
            "Perplexity",
            "Perplexity (normalized)",
            "Learnable scale (exp(log_scale))",
        ],
        "Protein": [
            config.protein.codebook_size,
            config.protein.latent_dim,
            f"{prot_metrics['mse_norm']:.6f}",
            f"{prot_metrics['mse_orig']:.6f}",
            f"{np.sqrt(prot_metrics['mse_orig']):.6f}",
            len(np.unique(prot_idx_np)),
            f"{len(np.unique(prot_idx_np)) / config.protein.codebook_size:.4f}",
            f"{prot_ppl:.1f}",
            f"{prot_ppl / config.protein.codebook_size:.4f}",
            f"{protein_vqvae.codebook.log_scale.exp().item():.4f}",
        ],
        "Ligand": [
            config.ligand.codebook_size,
            config.ligand.latent_dim,
            f"{lig_metrics['mse_norm']:.6f}",
            f"{lig_metrics['mse_orig']:.6f}",
            f"{np.sqrt(lig_metrics['mse_orig']):.6f}",
            len(np.unique(lig_idx_np)),
```



```

        f"{len(np.unique(lig_idx_np)) / config.ligand.codebook_size:.4f}"
        f"{lig_ppl:.1f}",
        f"{lig_ppl / config.ligand.codebook_size:.4f}",
        f"{ligand_vqvae.codebook.log_scale.exp().item():.4f}",
    ],
}
)
summary

```

	Metric	Protein	Ligand
0	Codebook size	2048	1024
1	Latent dim	16	8
2	Val MSE (normalized)	0.151219	0.002934
3	Val MSE (original scale)	1.685398	0.001459
4	Val RMSE (original scale)	1.298229	0.038194
5	Active codes	2048	1024
6	Utilization	1.0000	1.0000
7	Perplexity	1936.9	851.2
8	Perplexity (normalized)	0.9457	0.8313
9	Learnable scale (exp(log_scale))	1.0410	2.4858